Pydantic: Pythonにおけるデータ検証の簡素化

著者： Harrison Hoffman

公開日： 2024-04-10

情報源： https://realpython.com/python-pydantic/

Pydanticは、Python向けの強力なデータバリデーションおよび設定管理ライブラリであり、コードベースの堅牢性と信頼性を向上させるために設計されています。変数が整数であるかどうかを検証するような基本的なタスクから、複雑にネストされた辞書のキーや値が正しいデータ型を持っているかを確認するようなより高度なタスクまで、Pydanticは最小限のボイラープレートコードでほぼあらゆるデータバリデーションのシナリオを処理できます。

このチュートリアルで学ぶこと：

- Pydanticの `BaseModel` を使って**データスキーマ**を操作する方法
- 複雑なユースケース向けの**カスタムバリデータ**の作成方法
- Pydanticの `@validate_call` による**関数引数のバリデーション**方法
- `pydantic-settings` を使った設定管理と**アプリケーションの構成**方法

このチュートリアルを通じて、Pydanticの機能に関する実践的な例を学び、終了するころには、独自のバリデーションユースケースに対応するための確固たる基礎が身につきます。本チュートリアルを始めるにあたり、Pythonおよびオブジェクト指向プログラミングに関する中級レベルの理解があると、よりスムーズに理解できます。

> **コードの取得：** PydanticがPythonでのデータバリデーションをどのように簡素化するかを学ぶのに役立つ無料サンプルコードのダウンロードはこちら。

1 PythonのPydanticライブラリ

Pythonの大きな魅力の1つは、動的型付け言語であることです。動的型付けとは、コンパイル時に型を明示的に宣言する静的型付け言語とは異なり、実行時に変数の型が決定されることを意味します。動的型付けは迅速な開発や扱いやすさに優れている一方で、実運用アプリケーションでは、より堅牢な型チェックやデータバリデーションが必要になることがよくあります。そこで役立つのが、PythonのPydanticライブラリです。

Pydanticは急速に人気を集め、現在ではPythonで最も広く利用されているデータバリデーションライブラリとなっています。この最初のセクションでは、Pydanticの概要と、その強力な機能のプレビューを紹介します。また、Pydanticのインストール方法と、本チュートリアルに必要な追加依存関係についても解説します。

1.1 Pydanticに親しむ

Pydanticは、型ヒントを活用してデータスキーマの検証（バリデーション）とシリアライズを容易にする強力なPythonライブラリです。これにより、コードの堅牢性、可読性、簡潔性が向上し、デバッグも容易になります。また、Pydanticは多くの一般的な静的型チェッ

クツールやIDEと適切に連携するため、コードを実行する前にスキーマの問題を検出できます。

Pydanticの主な特長は以下のとおりです。

- **カスタマイズ性**：Pydanticで検証できるデータ型には、ほとんど制限がありません。Pythonのプリミティブ型から複雑にネストされたデータ構造まで、Pydanticを使えばほぼあらゆるPythonオブジェクトの検証とシリアライズが可能です。
- **柔軟性**：Pydanticを使用すると、データの検証時にどの程度厳密にするか、あるいは寛容にするかを制御できます。場合によっては、受け取ったデータを正しい型に強制変換（型キャスト）したいことがあります。例えば、本来浮動小数点数（`float`）であるべきデータが整数（`int`）として届いた場合にそれを受け入れるといったケースです。また別のケースでは、受け取るデータ型を厳密に強制したい場合もあるでしょう。Pydanticはどちらの調整にも対応しています。
- **シリアライズ**：Pydanticオブジェクトは辞書やJSON文字列としてシリアライズおよびデシリアライズが可能です。つまり、PydanticオブジェクトとJSONの相互変換をシームレスに行えます。この機能により、自己ドキュメント化されたAPIの作成や、JSON Schemaをサポートするほぼあらゆるツールとの連携が可能になっています。
- **パフォーマンス**：コアとなるバリデーションロジックがRustで書かれているため、Pydanticは非常に高速です。このパフォーマンス面での優位性により、特に大量のリクエストをスケーラブルに処理する必要があるREST APIのような高スループットなアプリケーションにおいて、迅速かつ信頼性の高いデータ処理を実現します。
- **エコシステムと業界での普及**：Pydanticは、FastAPI、LangChain、Polarsなど、多くの人気Pythonライブラリの依存関係として組み込まれています。また、大手IT企業の多くやその他の多様な業界でも広く利用されています。これは、Pydanticのコミュニティによるサポート、信頼性、そして堅牢性の証です。

これらは、Pydanticを魅力的なデータバリデーションライブラリにしている主要な機能のほんの一部です。本チュートリアルを通じて、これらが実際にどのように動作するかを確認していきます。次は、Pydanticおよびその各種依存関係のインストール方法について解説します。

1.2 Pydanticのインストール

PydanticはPyPIで公開されており、pipを使ってインストールできます。ターミナルまたはコマンドプロンプトを開き、新しい仮想環境を作成してから、以下のコマンドを実行してPydanticをインストールします。

```
(venv) $ python -m pip install pydantic
```

このコマンドにより、PyPIから最新バージョンのPydanticが環境にインストールされます。インストールが成功したことを確認するには、Python REPLを起動してPydanticをインポートします。

```
>>> import pydantic
```

インポートがエラーなしで実行できれば、Pydanticのインストールは成功であり、システムにPydanticのコア機能が導入された状態になります。

1.3 オプション依存関係の追加

Pydanticでは、オプションの依存関係もあわせてインストールできます。例えば、本チュートリアルではメールアドレスのバリデーションを扱うため、インストール時に以下の依存関

係を含めることができます。

```
(venv) $ python -m pip install "pydantic[email]"
```

また、Pydanticには設定管理用の独立したパッケージがあり、本チュートリアルでも取り上げます。これをインストールするには、以下のコマンドを実行します。

```
(venv) $ python -m pip install pydantic-settings
```

これで、本チュートリアルに必要なすべての依存関係がインストールされ、Pydanticの探索を始める準備が整いました。まずは、Pydanticにおけるデータスキーマの基本的な定義方法である「モデル」から見ていきましょう。

2 モデルの使用

Pydanticにおけるデータスキーマ定義の主な方法は、モデルを使用することです。Pydanticモデルは、Pythonのデータクラスに似たオブジェクトであり、アノテーションされたフィールドを用いてエンティティに関するデータを定義・保持します。データクラスとは異なり、Pydanticの主な焦点は自動的なデータの解析（パース）、バリデーション（検証）、およびシリアライズに置かれています。

これを理解する最良の方法は、実際に独自のモデルを作成してみることです。次にそれを行っていきましょう。

2.1 Pydantic BaseModelの操作

人事部門が従業員情報を管理するために使用するアプリケーションを構築しており、新しい従業員情報が正しい形式であることを検証する方法が必要であると仮定しましょう。例えば、各従業員にはID、名前、メールアドレス、生年月日、給与、部署、および福利厚生の選択情報が必要です。これはPydanticモデルにとって完璧なユースケースです！

従業員モデルを定義するには、Pydanticの `BaseModel` を継承するクラスを作成します。

```
# pydantic_models.py
from datetime import date
from uuid import UUID, uuid4
from enum import Enum
from pydantic import BaseModel, EmailStr

class Department(Enum):
    HR = "HR"
    SALES = "SALES"
    IT = "IT"
    ENGINEERING = "ENGINEERING"

class Employee(BaseModel):
    employee_id: UUID = uuid4()
    name: str
    email: EmailStr
    date_of_birth: date
    salary: float
    department: Department
    elected_benefits: bool
```

まず、従業員モデルの定義に必要な依存関係をインポートします。次に、社内のさまざまな部署を表す列挙型（enum）を作成し、これを使用して従業員モデルの department フィールドにアノテーションを付けます。

次に、BaseModel を継承するPydanticモデル Employee を定義し、アノテーションを介して従業員フィールドの名前と予期される型を定義します。以下は、Employee で定義した各フィールドの内訳と、Employee オブジェクトがインスタンス化される際にPydanticがそれをどのように検証するかを示しています。

- **employee_id**: 情報を保存したい従業員のUUIDです。UUID アノテーションを使用することで、Pydanticはこのフィールドが常に有効なUUIDであることを保証します。uuid4() を呼び出すように指定しているため、Employee の各インスタンスにはデフォルトでUUIDが割り当てられます。
- name: 従業員の名前です。Pydanticは文字列であることを予期します。
- email: Pydanticは内部でPythonのemail-validatorライブラリを使用し、各従業員の email が有効であることを確認します。
- **date_of_birth**: 各従業員の生年月日は、Pythonのdatetimeモジュールの date でアノテーションされているとおり、有効な日付である必要があります。date_of_birth に文字列を渡すと、Pydanticはそれを解析して date オブジェクトに変換しようと試みます。
- salary: 従業員の給与であり、浮動小数点数（float）であることが予期されます。
- **department**: 各従業員の部署は、Department enumで定義されている HR、SALES、IT、または ENGINEERING のいずれかでなければなりません。
- **elected_benefits**: このフィールドは従業員が福利厚生を選択したかどうかを保持し、Pydanticはブール値であることを予期します。

Employee オブジェクトを作成する最も簡単な方法は、他のPythonオブジェクトと同様にインスタンス化することです。これを行うには、PythonのREPLを開き、次のコードを実行します。

```
>>> from pydantic_models import Employee

>>> Employee(
...     name="Chris DeTuma",
...     email="cdetuma@example.com",
...     date_of_birth="1998-04-02",
...     salary=123_000.00,
...     department="IT",
...     elected_benefits=True,
... )
Employee(
    employee_id=UUID('73636d47-373b-40cd-a005-4819a84d9ea7'),
    name='Chris DeTuma',
    email='cdetuma@example.com',
    date_of_birth=datetime.date(1998, 4, 2),
    salary=123000.0,
    department=<Department.IT: 'IT'>,
    elected_benefits=True
)
```

このブロックでは、Employee をインポートし、必要な従業員フィールドをすべて指定してオブジェクトを作成します。Pydanticは渡されたフィールドのバリデーションと型変換（coercion）に成功し、有効な Employee オブジェクトを作成します。Pydanticが日付文字列を自動的に date オブジェクトに変換し、文字列 "IT" を対応する Department enum メンバに変換している点に注目してください。

次に、Employee インスタンスに無効なデータを渡そうとしたときに、Pydanticがどのように応答するかを見てみましょう。

```
>>> Employee(
...     employee_id="123",
...     name=False,
...     email="cdetumaexamplecom",
...     date_of_birth="1939804-02",
...     salary="high paying",
...     department="PRODUCT",
...     elected_benefits=300,
... )

Traceback (most recent call last):
pydantic_core._pydantic_core.ValidationError: 7 validation errors for
Employee

employee_id
  Input should be a valid UUID, invalid length: expected length 32 for
  simple format, found 3 [type=uuid_parsing, input_value='123',
  input_type=str] For further information visit
  https://errors.pydantic.dev/2.6/v/uuid_parsing

name
  Input should be a valid string [type=string_type, input_value=False,
  input_type=bool] For further information visit
  https://errors.pydantic.dev/2.6/v/string_type

email
  value is not a valid email address: The email address is not valid.
  It must have exactly one @-sign. [type=value_error,
  input_value='cdetumaexamplecom', input_type=str]

date_of_birth
  Input should be a valid date or datetime, invalid date separator,
  expected `-` [type=date_from_datetime_parsing,
  input_value='1939804-02', input_type=str] For further information
  visit https://errors.pydantic.dev/2.6/v/date_from_datetime_parsing

salary
  Input should be a valid number, unable to parse string as a number
  [type=float_parsing, input_value='high paying', input_type=str]
  For further information visit
  https://errors.pydantic.dev/2.6/v/float_parsing

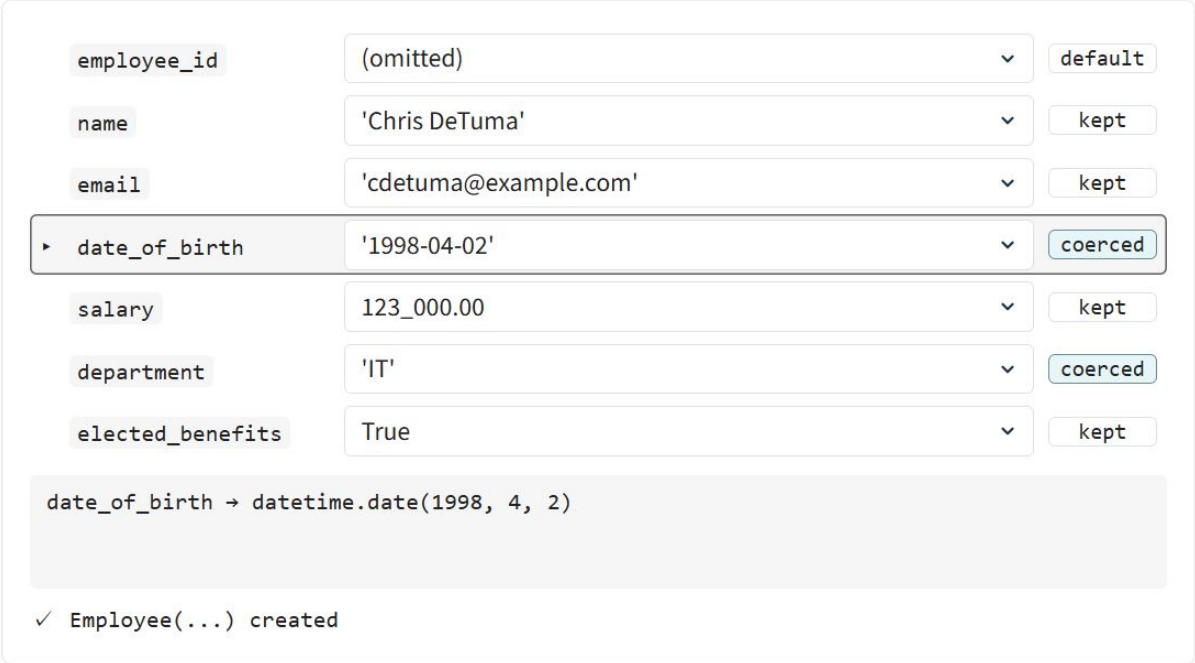
department
  Input should be 'HR', 'SALES', 'IT' or 'ENGINEERING'
  [type=enum, input_value='PRODUCT', input_type=str]

elected_benefits
  Input should be a valid boolean, unable to interpret input
  [type=bool_parsing, input_value=300, input_type=int]
  For further information visit
  https://errors.pydantic.dev/2.6/v/bool_parsing
```

この例では、無効なデータフィールドを持つ Employee オブジェクトを作成しました。Pydanticは各フィールドに対して詳細なエラーメッセージを表示し、何が期待されていたか、何が受け取られたか、そしてエラーの詳細情報を確認できる場所を示してくれます。

これをフィールドごとに調べるため、以下のウィジェットは記事の有効な例から始まりま

す。そこではPydanticが date_of_birth を date に、department を Department のメンバに強制変換する一方で、他のフィールドに渡された値はそのまま保持します。date_of_birth に対する '1939804-02' など、任意のフィールドで拒否される値を選択し、フッターが作成された Employee からバリデーションエラーに切り替わるのを確認してください。



この詳細なバリデーションは、Employee に無効なデータが保存されるのを防ぐため、非常に強力です。これにより、エラーなくインスタンス化された Employee オブジェクトには期待通りのデータが含まれているという安心感が得られ、コードのダウンストリーム（後続処理）や他のアプリケーションでこのデータを信頼できるようになります。

Pydanticの BaseModel には、辞書やJSONなどの他のオブジェクトからモデルを簡単に作成できるようにする一連のメソッドが備わっています。たとえば、辞書から Employee オブジェクトをインスタンス化したい場合は、.model_validate() クラスメソッドを使用できます。

```
>>> new_employee_dict = {
...     "name": "Chris DeTuma",
...     "email": "cdetuma@example.com",
...     "date_of_birth": "1998-04-02",
...     "salary": 123_000.00,
...     "department": "IT",
...     "elected_benefits": True,
... }

>>> Employee.model_validate(new_employee_dict)
Employee(
    employee_id=UUID('73636d47-373b-40cd-a005-4819a84d9ea7'),
    name='Chris DeTuma',
    email='cdetuma@example.com',
    date_of_birth=datetime.date(1998, 4, 2),
    salary=123000.0,
    department=<Department.IT: 'IT'>,
    elected_benefits=True
)
```

ここでは、従業員フィールドを持つ辞書 new_employee_dict を作成し、それを

.model_validate() に渡して Employee インスタンスを作成しています。内部では、Pydanticが各辞書エントリを検証して、期待されるデータに準拠していることを確認します。データに無効なものがある場合、Pydanticは先ほど見たのと同様にエラーをスローします。また、辞書に必要なフィールドが不足している場合も通知されます。

.model_validate_json() を使用すると、JSONオブジェクトに対しても同じことができます。

```
>>> new_employee_json = """
...  {"employee_id":"d2e7b773-926b-49df-939a-5e98cbb9c9eb",
...  "name":"Eric Slogrenta",
...  "email":"eslogrenta@example.com",
...  "date_of_birth":"1990-01-02",
...  "salary":125000.0,
...  "department":"HR",
...  "elected_benefits":false}
...  """

>>> new_employee = Employee.model_validate_json(new_employee_json)
>>> new_employee
Employee(
    employee_id=UUID('d2e7b773-926b-49df-939a-5e98cbb9c9eb'),
    name='Eric Slogrenta',
    email='eslogrenta@example.com',
    date_of_birth=datetime.date(1990, 1, 2),
    salary=125000.0,
    department=<Department.HR: 'HR'>,
    elected_benefits=False
)
```

この例では、new_employee_json は従業員フィールドを保持する有効なJSON文字列であり、.model_validate_json() を使用して new_employee_json から Employee オブジェクトを検証・作成しています。一見すると地味に思えるかもしれませんが、JSONからPydanticモデルを作成して検証できる機能は、JSONがWeb上でデータを転送する最も人気のある方法の1つであるため、非常に強力です。これが、FastAPIがREST APIの作成にPydanticに依存している理由の1つです。

Pydanticモデルを辞書やJSONとしてシリアライズすることもできます。

```
>>> new_employee.model_dump()
{
    'employee_id': UUID('d2e7b773-926b-49df-939a-5e98cbb9c9eb'),
    'name': 'Eric Slogrenta',
    'email': 'eslogrenta@example.com',
    'date_of_birth': datetime.date(1990, 1, 2),
    'salary': 125000.0,
    'department': <Department.HR: 'HR'>,
    'elected_benefits': False
}

>>> new_employee.model_dump_json()
'{"employee_id":"d2e7b773-926b-49df-939a-5e98cbb9c9eb",
  "name":"Eric Slogrenta",
  "email":"eslogrenta@example.com",
  "date_of_birth":"1990-01-02",
  "salary":125000.0,
  "department":"HR",
  "elected_benefits":false}'
```

ここでは、.model_dump() と .model_dump_json() を使用して、new_employee モデルをそれぞれ辞書とJSON文字列に変換しています。.model_dump_json() が date_of_birth と department を文字列として保持したJSONオブジェクトを返すことに注目してください。

Pydanticはすでにこれらのフィールドを検証してモデルをJSONに変換していますが、このJSONを後続で利用する側は、date_of_birth が有効な `date` である必要があることや、department が `Department` enum内のカテゴリである必要があることを知りません。これを解決するために、`Employee` モデルからJSON Schema（JSON スキーマ）を作成できます。

JSON Schemaは、JSONオブジェクトで期待されるフィールドと表現される値を教えてくれます。これは、`Employee` クラス定義のJSONバージョンと考えることができます。以下は、`Employee` のJSON Schemaを生成する方法です。

```
>>> Employee.model_json_schema()
{
    '$defs': {
        'Department': {
            'enum': ['HR', 'SALES', 'IT', 'ENGINEERING'],
            'title': 'Department',
            'type': 'string'
        }
    },
    'properties': {
        'employee_id': {
            'default': '73636d47-373b-40cd-a005-4819a84d9ea7',
            'format': 'uuid',
            'title': 'Employee Id',
            'type': 'string'
        },
        'name': {'title': 'Name', 'type': 'string'},
        'email': {
            'format': 'email',
            'title': 'Email',
            'type': 'string'
        },
        'date_of_birth': {
            'format': 'date',
            'title': 'Date Of Birth',
            'type': 'string'
        },
        'salary': {'title': 'Salary', 'type': 'number'},
        'department': {'$ref': '#/$defs/Department'},
        'elected_benefits': {'title': 'Elected Benefits', 'type': 'boolean'}
    },
    'required': [
        'name',
        'email',
        'date_of_birth',
        'salary',
        'department',
        'elected_benefits'
    ],
    'title': 'Employee',
    'type': 'object'
}
```

.model_json_schema() を呼び出すと、モデルのJSON Schemaを表す辞書が返されます。最初に見えるエントリは、department が取り得る値を示しています。また、フィールドがど

のようにフォーマットされるべきかに関する情報も確認できます。たとえば、このJSON Schemaによると、employee_id はUUIDであることが、date_of_birth は日付であることが期待されています。

JSON Schemaは json.dumps() を使用してJSON文字列に変換でき、これによりほぼすべてのプログラミング言語で、`Employee` モデルによって生成されたJSONオブジェクトを検証できるようになります。言い換えれば、Pydanticは入力データを検証してJSONとしてシリアライズできるだけでなく、JSON Schemaを通じて、他のプログラミング言語がモデルのデータを検証するために必要な情報も提供します。

これで、Pydanticの `BaseModel` を使用してデータを検証しシリアライズする方法が理解できました。次は、フィールドを使用してバリデーションをさらにカスタマイズする方法を学びます。

2.2 カスタマイズとメタデータのためのFieldの使用

ここまでで、`Employee` モデルは各フィールドのデータ型を検証し、`email`、date_of_birth、department などの一部のフィールドが有効な形式であることを確認するようになりました。しかし、さらに `salary` が正の数であること、`name` が空文字列でないこと、`email` に自社のドメイン名が含まれていることも確認したいとしましょう。これを達成するために、Pydanticの `Field` クラスを使用できます。

`Field` クラスを使用すると、モデルのフィールドをカスタマイズしたりメタデータを追加したりできます。これがどのように機能するかを確認するために、次の例を見てみましょう。

```
# pydantic_models.py
from datetime import date
from uuid import UUID, uuid4
from enum import Enum
from pydantic import BaseModel, EmailStr, Field

class Department(Enum):
    HR = "HR"
    SALES = "SALES"
    IT = "IT"
    ENGINEERING = "ENGINEERING"

class Employee(BaseModel):
    employee_id: UUID = Field(default_factory=uuid4, frozen=True)
    name: str = Field(min_length=1, frozen=True)
    email: EmailStr = Field(pattern=r".+@example\.com$")
    date_of_birth: date = Field(alias="birth_date", repr=False, frozen=True)
    salary: float = Field(alias="compensation", gt=0, repr=False)
    department: Department
    elected_benefits: bool
```

ここでは、以前使用した他の依存関係とともに `Field` をインポートし、`Employee` フィールドのいくつかにデフォルト値を割り当てています。以下は、フィールドに追加のバリデーションとメタデータを追加するために使用した `Field` パラメータの内訳です。

- **default_factory**: デフォルト値を生成する呼び出し可能オブジェクト（callable）を定義するために使用します。上の例では、default_factory を `uuid4` に設定しています。これにより、必要に応じて uuid4() が呼び出され、employee_id のランダムなUUIDが生成されます。より柔軟性を高めるために、ラムダ（lambda）関数を使用することもできます。

- frozen: フィールドをイミュータブル（不可変）にするために設定できるブールパラメータです。これは、frozen が True に設定されると、モデルがインスタンス化された後に該当するフィールドを変更できなくなることを意味します。この例では、frozen パラメータを使用して employee_id、name、および date_of_birth をイミュータブルにしています。
- **min_length**: min_length と max_length を使用して文字列フィールドの長さを制御できます。上の例では、name が少なくとも1文字以上であることを保証しています。
- pattern: 文字列フィールドの場合、pattern に正規表現を指定して、そのフィールドに期待されるパターンに一致させることができます。たとえば、上の例で email に対して正規表現を使用すると、Pydanticはすべてのメールアドレスが @example.com で終わることを保証します。
- alias: フィールドにエイリアス（別名）を割り当てたい場合にこのパラメータを使用できます。たとえば、date_of_birth を birth_date として呼び出したり、salary を compensation として呼び出したりすることを許可できます。これらのエイリアスは、モデルをインスタンス化またはシリアライズするときに使用できます。
- gt: 「greater than（より大きい）」の略であるこのパラメータは、数値フィールドの最小値を設定するために使用されます。この例では、gt=0 を設定することで salary が常に正の数であることを保証します。Pydanticには、「less than（より小さい）」の略である lt など、その他の数値制約も用意されています。
- repr: このブールパラメータは、モデルのフィールド表現（repr）にフィールドが表示されるかどうかを決定します。この例では、Employee インスタンスを出力したときに date_of_birth や salary は表示されません。

この追加のバリデーションが動作している様子を確認するために、不適切なデータで Employee モデルを作成しようとしたときに何が起こるか注目してください。

```
>>> from pydantic_models import Employee

>>> incorrect_employee_data = {
...     "name": "",
...     "email": "cdetuma@fakedomain.com",
...     "birth_date": "1998-04-02",
...     "salary": -10,
...     "department": "IT",
...     "elected_benefits": True,
... }

>>> Employee.model_validate(incorrect_employee_data)
Traceback (most recent call last):

pydantic_core._pydantic_core.ValidationError: 3 validation errors for
Employee
name
  String should have at least 1 character [type=string_too_short,
   input_value='', input_type=str] For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
email
  String should match pattern '.+@example\.com$'
  [type=string_pattern_mismatch,
  input_value='cdetuma@fakedomain.com', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_pattern_mismatch
salary
```

```
  Input should be greater than 0 [type=greater_than, input_value=-10,
  input_type=int] For further information visit
  https://errors.pydantic.dev/2.6/v/greater_than
```

ここでは、更新された Employee モデルをインポートし、不適切なデータを持つ辞書を検証しようとしています。それに対してPydanticは、name は少なくとも1文字必要であること、email は自社のドメイン名に一致する必要があること、salary は0より大きくする必要があることという3つのバリデーションエラーを返します。

次に、正解の Employee データを検証したときに得られる追加機能に注目してください。

```
>>> employee_data = {
...     "name": "Clyde Harwell",
...     "email": "charwell@example.com",
...     "birth_date": "2000-06-12",
...     "compensation": 100_000,
...     "department": "ENGINEERING",
...     "elected_benefits": True,
... }

>>> employee = Employee.model_validate(employee_data)
>>> employee
Employee(
    employee_id=UUID('614c6f75-8528-4272-9cfc-365ddfafebd9'),
    name='Clyde Harwell',
    email='charwell@example.com',
    department=<Department.ENGINEERING: 'ENGINEERING'>,
    elected_benefits=True)

>>> employee.salary
100000.0

>>> employee.date_of_birth
datetime.date(2000, 6, 12)
```

このブロックでは、辞書を作成し、.model_validate() で Employee モデルを作成します。employee_data 内で、date_of_birth の代わりに birth_date を、salary の代わりに compensation を使用していることに注目してください。Pydanticはこれらのエイリアスを認識し、内部で正しいフィールド名にそれらの値を割り当てます。

repr=False を設定したため、salary と date_of_birth が Employee の表現内に表示されていないことがわかります。それらの値を確認するには、属性として明示的にアクセスする必要があります。最後に、フリーズされた（frozen）フィールドを変更しようとしたときに何が起こるか確認してください。

```
>>> employee.department = "HR"
>>> employee.name = "Andrew TuGrendele"
Traceback (most recent call last):

pydantic_core._pydantic_core.ValidationError: 1
validation error for Employee
name
  Field is frozen [type=frozen_field, input_value='Andrew TuGrendele',
  input_type=str]
  For further information visit
  https://errors.pydantic.dev/2.6/v/frozen_field
```

ここでは、まず department の値を IT から HR に変更します。department はフリーズされたフィールドではないため、これは完全に問題ありません。しかし、name を変更しようとすると、Pydanticは name がフリーズされたフィールドであるというエラーを出力します。

これで、Pydanticの BaseModel クラスと Field クラスをしっかり理解できました。これらだけでもデータスキーマにさまざまなバリデーションルールやメタデータを定義できますが、これだけでは不十分な場合もあります。次は、Pydanticのバリデータを使用してフィールドバリデーションをさらに押し進める方法を学びます。

3 バリデータの操作

ここまでは、Pydanticの BaseModel を使用して事前定義された型でモデルフィールドを検証し、さらに Field を組み込んでバリデーションをカスタマイズしてきました。BaseModel と Field だけでも多くのケースに対応できますが、カスタムロジックを必要とするより複雑なバリデーションシナリオでは、Pydanticのバリデータを使用する必要があります。

バリデータを使用すると、関数で表現できるほぼあらゆるバリデーションロジックを実行できます。次にその方法を見ていきましょう。

3.1 モデルとフィールドの検証

従業員の例を続けると、あなたの会社には18歳以上の従業員のみを採用するというポリシーがあるとします。新しい Employee オブジェクトを作成するときは常に、その従業員が18歳以上であることを確認する必要があります。これを処理するために、age フィールドを追加して Field クラスを使用し、従業員が18歳以上であることを強制することもできます。しかし、すでに従業員の生年月日を保存しているため、これは冗長に思えます。

より良い解決策は、Pydanticのフィールドバリデータを使用することです。フィールドバリデータを使用すると、モデルにクラスメソッドを追加することで、BaseModel フィールドにカスタムのバリデーションロジックを適用できます。すべての従業員が18歳以上であることを強制するために、Employee モデルに以下のフィールドバリデータ（Field validator）を追加できます。

```
# pydantic_models.py
from datetime import date
from uuid import UUID, uuid4
from enum import Enum
from pydantic import BaseModel, EmailStr, Field, field_validator

class Department(Enum):
    HR = "HR"
    SALES = "SALES"
    IT = "IT"
    ENGINEERING = "ENGINEERING"

class Employee(BaseModel):
    employee_id: UUID = Field(default_factory=uuid4, frozen=True)
    name: str = Field(min_length=1, frozen=True)
    email: EmailStr = Field(pattern=r".+@example\.com$")
    date_of_birth: date = Field(alias="birth_date", repr=False, frozen=True)
    salary: float = Field(alias="compensation", gt=0, repr=False)
    department: Department
    elected_benefits: bool

    @field_validator("date_of_birth")
    @classmethod
```

```
    def check_valid_age(cls, date_of_birth: date) -> date:
        today = date.today()
        eighteen_years_ago = date(today.year - 18, today.month, today.day)

        if date_of_birth > eighteen_years_ago:
            raise ValueError("Employees must be at least 18 years old.")

        return date_of_birth
```

このブロックでは、field_validator をインポートし、それを使用して Employee 内の .check_valid_age() というクラスメソッドをデコレートしています。フィールドバリデータはクラスメソッドとして定義する必要があります。.check_valid_age() 内では、本日の日付から18年前の日付を計算しています。従業員の date_of_birth がその日付よりも後の場合、エラーが発生します。

このバリデータがどのように機能するかを確認するには、次の例を参照してください。

```
>>> from pydantic_models import Employee
>>> from datetime import date, timedelta

>>> young_employee_data = {
...     "name": "Jake Bar",
...     "email": "jbar@example.com",
...     "birth_date": date.today() - timedelta(days=365 * 17),
...     "compensation": 90_000,
...     "department": "SALES",
...     "elected_benefits": True,
... }

>>> Employee.model_validate(young_employee_data)
Traceback (most recent call last):

pydantic_core._pydantic_core.ValidationError:
1 validation error for Employee
birth_date
  Value error, Employees must be at least 18 years old.
  [type=value_error, input_value=datetime.date(2007, 4, 10),
  input_type=date]
  For further information visit
  https://errors.pydantic.dev/2.6/v/value_error
```

この例では、現在の日付から17年前の birth_date を指定しています。young_employee_data を検証するために .model_validate() を呼び出すと、従業員は少なくとも18歳でなければならないというエラーが発生します。

想像通り、Pydanticの field_validator() を使用すると、フィールドのバリデーションを自由にカスタマイズできます。ただし、複数のフィールドを相互に比較したい場合や、モデル全体を検証したい場合、field_validator() は機能しません。そのためには、モデルバリデータを使用する必要があります。

例として、あなたの会社ではIT部門では契約社員のみを採用しているとします。このため、IT部門の労働者は福利厚生を受ける資格がなく、その elected_benefits フィールドは False である必要があります。Pydanticの model_validator() を使用して、この制約を強制できます。

```
# pydantic_models.py
from typing import Self
```

```
from datetime import date
from uuid import UUID, uuid4
from enum import Enum
from pydantic import (
    BaseModel,
    EmailStr,
    Field,
    field_validator,
    model_validator,
)

class Department(Enum):
    HR = "HR"
    SALES = "SALES"
    IT = "IT"
    ENGINEERING = "ENGINEERING"

class Employee(BaseModel):
    employee_id: UUID = Field(default_factory=uuid4, frozen=True)
    name: str = Field(min_length=1, frozen=True)
    email: EmailStr = Field(pattern=r".+@example\.com$")
    date_of_birth: date = Field(alias="birth_date", repr=False, frozen=True)
    salary: float = Field(alias="compensation", gt=0, repr=False)
    department: Department
    elected_benefits: bool

    @field_validator("date_of_birth")
    @classmethod
    def check_valid_age(cls, date_of_birth: date) -> date:
        today = date.today()
        eighteen_years_ago = date(today.year - 18, today.month, today.day)

        if date_of_birth > eighteen_years_ago:
            raise ValueError("Employees must be at least 18 years old.")

        return date_of_birth

    @model_validator(mode="after")
    def check_it_benefits(self) -> Self:
        department = self.department
        elected_benefits = self.elected_benefits

        if department == Department.IT and elected_benefits:
            raise ValueError(
                "IT employees are contractors and don't qualify for benefits"
            )
        return self
```

ここで、PythonのSelf 型とPydanticの `model_validator()` をインポートに追加します。次に、従業員がIT部門に所属しており `elected_benefits` フィールドが `True` である場合にエラーを発生させるメソッド `.check_it_benefits()` を作成します。`@model_validator` で `mode` を `"after"` に設定すると、Pydanticはモデルがインスタンス化された後に `.check_it_benefits()` を実行します。

> 注: `.check_it_benefits()` がPythonの `Self` 型でアノテーションされていることに気付いたかもしれません。これは、`.check_it_benefits()` が `Employee` クラスのインスタンスを返すためであり、`Self` 型がこれに対して推奨されるア

ノテーションだからです。3.11より前のPythonバージョンを使用している場合は、typing_extensions から Self 型をインポートする必要があります。

新しいモデルバリデータの動作を確認するには、次の例を参照してください。

```
>>> from pydantic_models import Employee

>>> new_employee = {
...     "name": "Alexis Tau",
...     "email": "ataue@example.com",
...     "birth_date": "2001-04-12",
...     "compensation": 100_000,
...     "department": "IT",
...     "elected_benefits": True,
... }

>>> Employee.model_validate(new_employee)
Traceback (most recent call last):

pydantic_core._pydantic_core.ValidationError: 1 validation error for
Employee
  Value error, IT employees are contractors and don't qualify for
  benefits.
  [type=value_error, input_value={'name': 'Alexis Tau',
  ...elected_benefits': True},
  input_type=dict]
    For further information visit
    https://errors.pydantic.dev/2.6/v/value_error
```

この例では、IT 部門で elected_benefits を True に設定した Employee モデルを作成しようとしています。.model_validate() を呼び出すと、Pydanticはエラーをスローし、IT部門の従業員は契約社員であるため福利厚生の資格がないことを伝えます。

モデルバリデータとフィールドバリデータを使用すると、思いつく限りのほぼあらゆるカスタムバリデーションを実装できます。これで、独自のユースケースに合わせてPydanticモデルを作成するための堅牢な基礎ができたはずです。次は、話題を変えて、BaseModel のフィールドだけでなく、任意の関数を検証するためにPydanticを使用する方法を見ていきます。

3.2 バリデーションデコレータを使用した関数の検証

BaseModel はデータスキーマを検証するためのPydanticの主力クラスですが、@validate_call デコレータを使用して関数の引数を検証することもできます。これにより、バリデーションロジックを手動で実装することなく、分かりやすい型エラーを備えた堅牢な関数を作成できます。

この仕組みを確認するために、クライアントが購入を行った後に請求書を送信する関数を書いていると仮定します。この関数は、クライアントの名前、メールアドレス、購入したアイテム、および請求総額を受け取り、メールを構築して送信します。入力が誤っていると、メールが送信されなかったり、フォーマットが崩れたり、クライアントに誤った請求がなされたりする可能性があるため、これらの入力をすべて検証する必要があります。

これを実現するために、以下の関数を記述します。

```
# validate_functions.py
import time
from typing import Annotated
from pydantic import PositiveFloat, Field, EmailStr, validate_call
```

```
@validate_call
def send_invoice(
    client_name: Annotated[str, Field(min_length=1)],
    client_email: EmailStr,
    items_purchased: list[str],
    amount_owed: PositiveFloat,
) -> str:

    email_str = f"""
    Dear {client_name}, \n
    Thank you for choosing xyz inc! You
    owe ${amount_owed:,.2f} for the following items: \n
    {items_purchased}
    """

    print(f"Sending email to {client_email}...")
    time.sleep(2)

    return email_str
```

まず、send_invoice() の記述とアノテーションに必要な依存関係をインポートします。次に、@validate_call でデコレートされた send_invoice() を作成します。send_invoice() を実行する前に、@validate_call は各入力がアノテーションに準拠していることを確認します。この場合、@validate_call は client_name が1文字以上であるか、client_email が正しくフォーマットされているか、items_purchased が文字列のリストであるか、amount_owed が正の浮動小数点数であるかをチェックします。

入力の1つがアノテーションに準拠していない場合、Pydanticは BaseModel で既に見たようなエラーをスローします。すべての入力が有効な場合、send_invoice() は文字列を作成し、time.sleep(2) を使用してクライアントへの送信をシミュレートします。

> **注**: client_name がPythonの Annotated 型でアノテーションされていることに気付いたかもしれません。一般に、関数の引数に関するメタデータを提供したい場合に Annotated を使用できます。Pydanticは、Field によって指定されたメタデータを持つ関数の引数を検証する必要がある場合、Annotated を使用することを推奨しています。
>
> ただし、default_factory を使用して関数の引数にデフォルト値を割り当てる場合は、引数を Field インスタンスに直接割り当てる必要があります。この例はPydanticのドキュメントで確認できます。

@validate_call と send_invoice() の実際の動作を確認するには、新しい Python REPL を開いて次のコードを実行します。

```
>>> from validate_functions import send_invoice

>>> send_invoice(
...     client_name="",
...     client_email="ajolawsonfakedomain.com",
...     items_purchased=["pie", "cookie", 17],
...     amount_owed=0,
... )
Traceback (most recent call last):
pydantic_core._pydantic_core.ValidationError: 4 validation errors for
send_invoice
client_name
  String should have at least 1 character [type=string_too_short,
```

```
  input_value='', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
client_email
  value is not a valid email address: The email address is not valid.
  It must have exactly one @-sign. [type=value_error,
  input_value='ajolawsonfakedomain.com', input_type=str]
items_purchased.2
  Input should be a valid string [type=string_type, input_value=17,
  input_type=int]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_type
amount_owed
  Input should be greater than 0 [type=greater_than, input_value=0,
  input_type=int]
    For further information visit
    https://errors.pydantic.dev/2.6/v/greater_than
```

この例では、send_invoice() をインポートし、無効な関数引数を渡しています。Pydanticの @validate_call はこれを認識し、client_name には少なくとも1文字が必要であること、client_email が無効であること、items_purchased には文字列が含まれるべきであること、amount_owed は0より大きくあるべきであることを示すエラーをスローします。

有効な入力を渡すと、send_invoice() は期待通りに実行されます。

```
>>> email_str = send_invoice(
...     client_name="Andrew Jolawson",
...     client_email="ajolawson@fakedomain.com",
...     items_purchased=["pie", "cookie", "cake"],
...     amount_owed=20,
... )
Sending email to ajolawson@fakedomain.com...

>>> print(email_str)

    Dear Andrew Jolawson,

    Thank you for choosing xyz inc! You
    owe $20.00 for the following items:

    ['pie', 'cookie', 'cake']
```

@validate_call は BaseModel ほど柔軟ではありませんが、関数の引数に強力なバリデーションを適用するために使用できます。これにより、多くの時間を節約でき、ボイラープレートな型チェックやバリデーションロジックの記述を避けることができます。これまでにこれを実行したことがある方なら、関数の引数ごとにassert 文を書くことがどれほど面倒かお分かりでしょう。多くのユースケースにおいて、@validate_call がこれを代わりに処理してくれます。

このチュートリアルの最後のセクションでは、設定管理やコンフィグレーションにPydanticを使用する方法を学びます。

4 設定の管理

Pythonアプリケーションを設定する最も一般的な方法の1つは環境変数を使用することです。環境変数とは、Pythonコードの外部、オペレーティングシステム上に存在する変数であ

り、コードや他のプログラムから読み取ることができます。環境変数として保存したいデータの例には、シークレットキー、データベースの認証情報、API認証情報、サーバーアドレス、アクセストークンなどが含まれます。

環境変数は開発環境と本番環境の間で頻繁に変更され、その多くには機密情報が含まれています。このため、コード内で環境変数を解析、検証、統合するための堅牢な方法が必要です。これは pydantic-settings の最適なユースケースであり、このセクションで詳しく見ていきます。

4.1 BaseSettings を使用したアプリケーションの設定

pydantic-settings はPythonで環境変数を管理するための最も強力な方法の1つであり、FastAPI などの人気ライブラリでも広く採用・推奨されています。pydantic-settings を使用すると、BaseModel に類似した、環境変数を解析および検証するモデルを作成できます。

pydantic-settings のメインクラスは BaseSettings であり、BaseModel と同じ機能をすべて備えています。ただし、BaseSettings を継承するモデルを作成した場合、モデルの初期化処理はキーワード引数として渡されなかったフィールドを環境変数から読み取ろうとします。

色がどのように機能するかを確認するために、アプリケーションがデータベースと別のAPIサービスに接続すると仮定します。データベースの認証情報とAPIキーは時間の経過とともに変更される可能性があり、デプロイする環境によっても異なることがよくあります。これを処理するには、次のような BaseSettings モデルを作成できます。

```
# settings_management.py
from pydantic import HttpUrl, Field
from pydantic_settings import BaseSettings

class AppConfig(BaseSettings):
    database_host: HttpUrl
    database_user: str = Field(min_length=5)
    database_password: str = Field(min_length=10)
    api_key: str = Field(min_length=20)
```

このスクリプトでは、BaseSettings モデルの作成に必要な依存関係をインポートしています。pydantic_settings から BaseSettings をインポートする際、ハイフンではなくアンダースコアを使用している点に注意してください。次に、BaseSettings を継承し、データベースとAPIキーに関するフィールドを保持するモデル AppConfig を定義します。この例では、database_host は有効なHTTP URLである必要があり、残りのフィールドには最小長の制約があります。

次に、ターミナルを開き、以下の環境変数を追加します。Linux、macOS、またはWindows Bashを使用している場合は、export コマンドでこれを行うことができます。

```
(venv) $ export DATABASE_HOST="http://somedatabaseprovider.us-east-2.com"
(venv) $ export DATABASE_USER="username"
(venv) $ export DATABASE_PASSWORD="asdfjl348ghl@9fhsl4"
(venv) $ export API_KEY="ajfsdla48fsdal49fj94jf93-f9dsal"
```

また、Windows PowerShellで環境変数を設定することもできます。その後、新しいPython REPLを開き、AppConfig をインスタンス化できます。

```
>>> from settings_management import AppConfig

>>> AppConfig()
```

```
AppConfig(
    database_host=Url('http://somedatabaseprovider.us-east-2.com/'),
    database_user='username',
    database_password='asdfjl348ghl@9fhsl4',
    api_key='ajfsdla48fsdal49fj94jf93-f9dsal'
)
```

AppConfig をインスタンス化する際にフィールド名を何も指定していないことに注目してください。代わりに、BaseSettings モデルは設定した環境変数からフィールドを読み取ります。また、環境変数をすべて大文字でエクスポートしたにもかかわらず、AppConfig がそれらを正常に解析して保存したことにも注目してください。これは、BaseSettings が環境変数とフィールド名のマッチングにおいて大文字小文字を区別しないためです。

次に、Python REPLを閉じ、無効な環境変数を作成します。

```
(venv) $ export DATABASE_HOST="somedatabaseprovider.us-east-2"
(venv) $ export DATABASE_USER="usee"
(venv) $ export DATABASE_PASSWORD="asdf"
(venv) $ export API_KEY="ajf"
```

ここで別のPython REPLを開き、AppConfig を再度インスタンス化します。

```
>>> from settings_management import AppConfig

>>> AppConfig()
Traceback (most recent call last):
pydantic_core._pydantic_core.ValidationError: 4 validation errors for
AppConfig
database_host
  Input should be a valid URL, relative URL without a base
  [type=url_parsing, input_value='somedatabaseprovider.us-east-2',
  input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/url_parsing
database_user
  String should have at least 5 characters [type=string_too_short,
  input_value='usee', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
database_password
  String should have at least 10 characters [type=string_too_short,
  input_value='asdf', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
api_key
  String should have at least 20 characters [type=string_too_short,
  input_value='ajf', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
```

今回、AppConfig をインスタンス化しようとすると、pydantic-settings は database_host が有効なURLではないこと、および残りのフィールドが最小長の制約を満たしていないことを示すエラーをスローします。

これは単純化された設定例ですが、BaseSettings を活用すれば環境変数から必要なほぼすべてのデータを解析および検証できます。BaseModel で実行できる検証は、モデルバリデー

タやフィールドバリデータによるカスタム検証を含め、すべて BaseSettings でも実行できます。

最後に、SettingsConfigDict を使用して BaseSettings の挙動をさらにカスタマイズする方法を学びます。

4.2 SettingsConfigDict を使用した設定のカスタマイズ

前の例では、環境変数を解析・検証する BaseSettings モデルの基本的な例を見ました。しかし、BaseSettings モデルの挙動をさらにカスタマイズしたい場合もあるでしょう。これは SettingsConfigDict を使用して実現できます。

よくあるケースとして、各環境変数を手動でエクスポートできず、.env ファイルから読み取る必要がある場合を考えます。解析時に BaseSettings が大文字小文字を区別するようにし、モデルで指定した変数以外の不要な環境変数が .env ファイルに含まれないようにしたいとします。SettingsConfigDict を使用すると、以下のように記述できます。

```
# settings_management.py
from pydantic import HttpUrl, Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class AppConfig(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=True,
        extra="forbid",
    )

    database_host: HttpUrl
    database_user: str = Field(min_length=5)
    database_password: str = Field(min_length=10)
    api_key: str = Field(min_length=20)
```

このスクリプトは前の例と同じですが、今回は SettingsConfigDict をインポートし、AppConfig 内で初期化しています。SettingsConfigDict 内では、環境変数を .env ファイルから読み取ること、大文字小文字の区別を強制すること、および .env ファイル内の余分な環境変数を禁止することを指定しています。

次に、settings_management.py と同じディレクトリに .env という名前のファイルを作成し、以下の環境変数を記述します。

```
database_host=http://somedatabaseprovider.us-east-2.com/
database_user=username
database_password=asdfjfffff1348ghl@9fhsl4
api_key=ajfsdla48fsdal49fj94jf93-f9dsal
```

これで、Python REPLを開いて AppConfig モデルを初期化できます。

```
>>> from settings_management import AppConfig

>>> AppConfig()
AppConfig(
    database_host=Url('http://somedatabaseprovider.us-east-2.com/'),
    database_user='username',
    database_password='asdfjfffff1348ghl@9fhsl4',
    api_key='ajfsdla48fsdal49fj94jf93-f9dsal'
)
```

ご覧のとおり、AppConfig は .env ファイル内の環境変数を正常に解析・検証しました。

最後に、無効な変数を .env ファイルに追加してみましょう。

```
DATABASE_HOST=http://somedatabaseprovider.us-east-2.com/
database_user=username
database_password=asdfjfffffl348ghl@9fhsl4
api_key=ajfsdla48fsdal49fj94jf93-f9dsal
extra_var=shouldntbehere
```

ここでは、database_host を DATABASE_HOST に変更して大文字小文字の区別制約に違反させ、さらに存在すべきでない余分な環境変数を追加しました。これを検証しようとしたときのモデルのレスポンスは以下のようになります。

```
>>> from settings_management import AppConfig

>>> AppConfig()
Traceback (most recent call last):
pydantic_core._pydantic_core.ValidationError: 3 validation errors for
AppConfig
database_host
  Field required [type=missing, input_value={'database_user':
  'userna..._var': 'shouldntbehere'}, input_type=dict]
    For further information visit
    https://errors.pydantic.dev/2.6/v/missing
DATABASE_HOST
  Extra inputs are not permitted [type=extra_forbidden,
  input_value='http://somedatabaseprovider.us-east-2.com/', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/extra_forbidden
extra_var
  Extra inputs are not permitted [type=extra_forbidden,
  input_value='shouldntbehere', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/extra_forbidden
```

database_host が不足していること、および .env ファイル内に余分な環境変数があることを示すエラーの一覧が表示されます。大文字小文字の区別制約があるため、モデルは extra_var だけでなく DATABASE_HOST も余分な変数として認識している点に注目してください。

SettingsConfigDict やより全般的な BaseSettings で実行できることは他にもたくさんありますが、これらの例によって、自身のユースケースに合わせて pydantic-settings で環境変数を管理する方法のアイデアが得られたはずです。

5 まとめ

Pydanticは、使いやすく、高速で、広く信頼されているPythonのデータ検証ライブラリです。これでPydanticの広範な概要を把握できたので、自身のプロジェクトでPydanticを使い始めるために必要な知識とリソースが揃いました。

このチュートリアルで学んだこと:

- **Pydantic** とは何か、そしてなぜこれほど広く採用されているのか
- Pydanticを **インストール** する方法

- `BaseModel` と **バリデータ** を使ってデータスキーマを解析、検証、シリアライズする方法
- `@validate_call` を使って関数のカスタム検証ロジックを記述する方法
- `pydantic-settings` を使って環境変数を解析・検証する方法

Pydanticはコードの堅牢性と信頼性を高め、Pythonの使いやすさと静的型付け言語の組み込みデータ検証との間のギャップを部分的に埋めます。想定し得るほぼあらゆるデータ解析、検証、シリアライズのユースケースに対して、Pydanticはエレガントなソリューションを提供します。

一度に1つのレコードではなくデータテーブル全体をチェックする必要がある場合は、PythonでPointblankを使用してデータを検証するを参照してください。

> **コードを入手:** PydanticがPythonでのデータ検証をどのように簡素化できるかを学ぶのに役立つ無料のサンプルコードをダウンロードするにはここをクリックしてください。